Team Sentinel: Wazuh Attack Monitoring Report

23 October 2026 · SOC Analyst Bootcamp

Summary

We discovered that Wazuh sees an account-creation and privilege-escalation attack on both machines even with default logging, but it tells the full story only when Sysmon and Auditd are added. We ran the same attack twice and read the results in the Wazuh dashboard, not on the terminals.

- **Run 1, default logging:** Wazuh alerted on the new accounts and the group changes (the Administrators change at level 12), but gave no command line and only a generic level 3 alert for the sudo and shadow steps.
- **Run 2, Sysmon and Auditd:** Wazuh showed the command line, the parent process, the file hash, and which program touched `/etc/shadow` or ran `usermod`.
- **Our answer:** PARTIALLY. Both tools add real detail, but the Linux rule needs tuning and Auditd did not show `useradd` (see the conclusion).

Objective and scope

We set out to answer one question: if someone creates a user account and raises its privileges, does the SOC see it in Wazuh, and how much detail does it get?

- **Scenario:** suspicious user creation followed by privilege escalation, on a Windows 10 machine and an Ubuntu server.
- **Run 1:** default logging only.
- **Run 2:** the same attack with Sysmon (Windows) and Auditd (Linux) added.
- **Output:** the evidence from the dashboard, the visibility gaps, and a final YES, PARTIALLY or NO answer.

Environment and method

We built the lab on two VirtualBox machines on a host-only network, with Wazuh 4.14 collecting from both.

Role	Machine	Notes
Wazuh server	Ubuntu Server `wazuh-anselm`	Manager, indexer and dashboard in one machine; also monitored as its own agent
Windows endpoint	`WIN-AGENT` (agent 001, 192.168.56.102)	Windows 10 Pro, agent v4.14.9

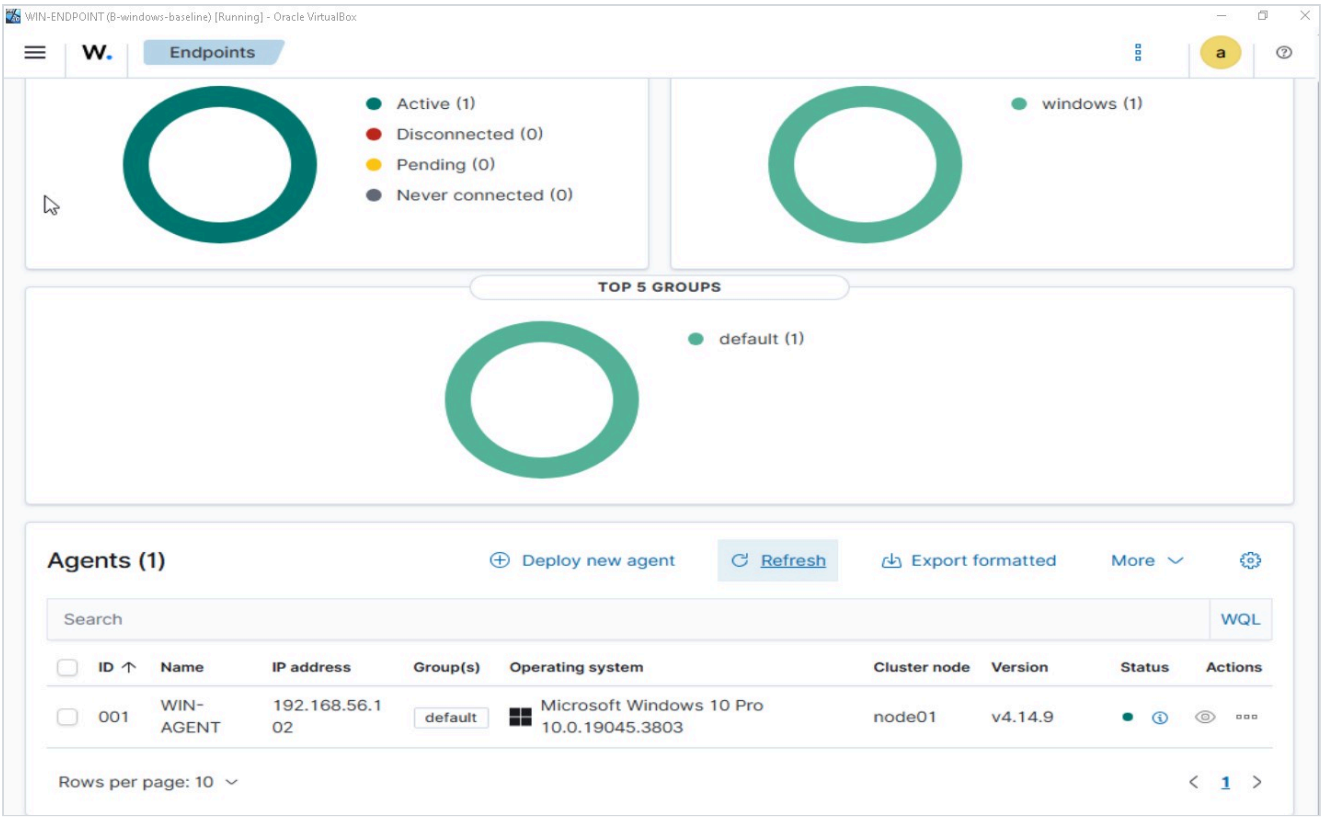


Figure 1: WIN-AGENT is Active on the Wazuh Endpoints page.

Run 1 used default logging: Windows Security events, with four audit policies switched on (User Account Management, Security Group Management, Logon and Process Creation), and Ubuntu's normal system and authentication logs.

Run 2 added Sysmon on Windows and Auditd on Ubuntu. We also wrote one custom Wazuh rule for Auditd (rule 100100, level 8), because Wazuh's built-in audit rules do not raise alerts on their own.

Time. The dashboard shows event times about 8 hours earlier than the machine clocks. This report quotes the machine clock (WAT).

Run 1: what Wazuh showed with default logging

Each finding below was read in the Wazuh dashboard (Threat Hunting, Events). Times are the machine clock on 21 Oct 2026.

A new Windows account was created

We discovered that a new account, `svc_backup`, was created on WIN-AGENT at 14:27:13. Wazuh raised rule 60109, "User account enabled or created", level 8 (Windows event 4720).

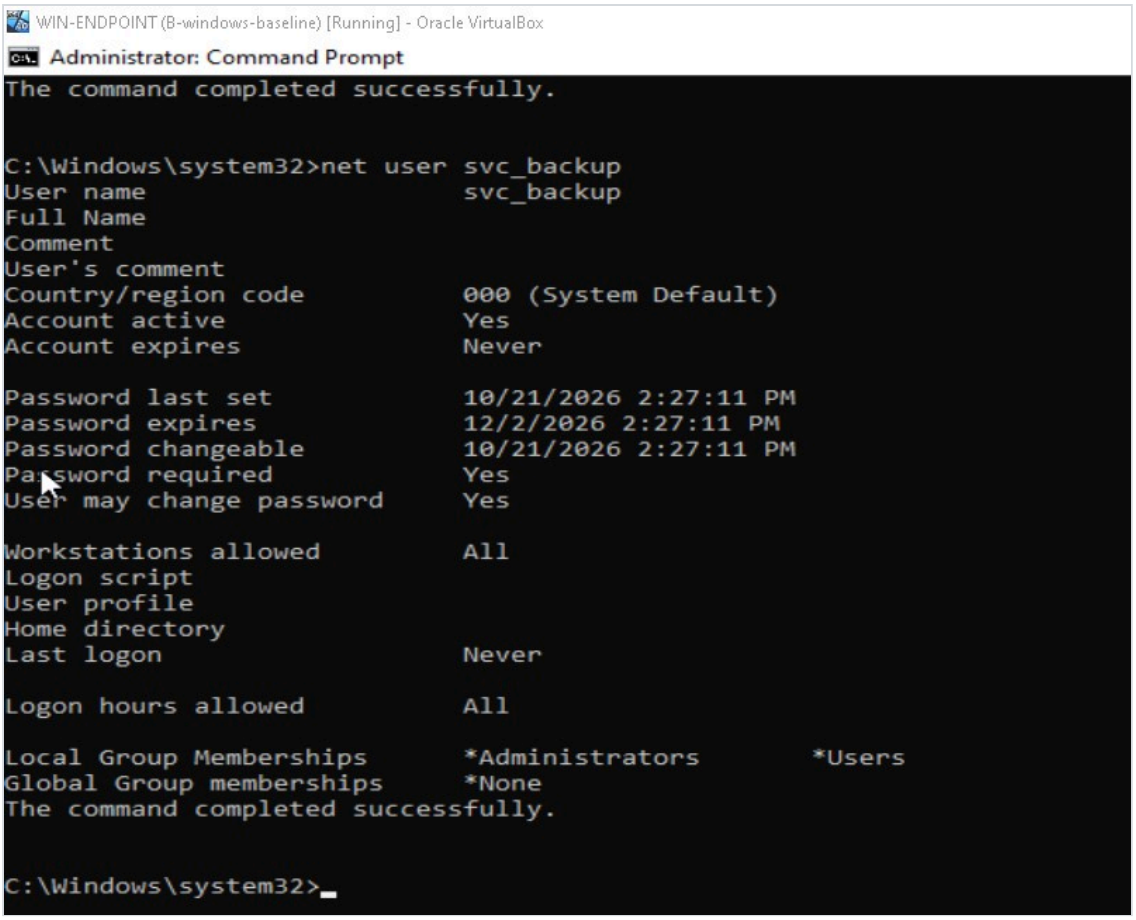


Figure 2: the account on the Windows machine, a member of Administrators and Users.

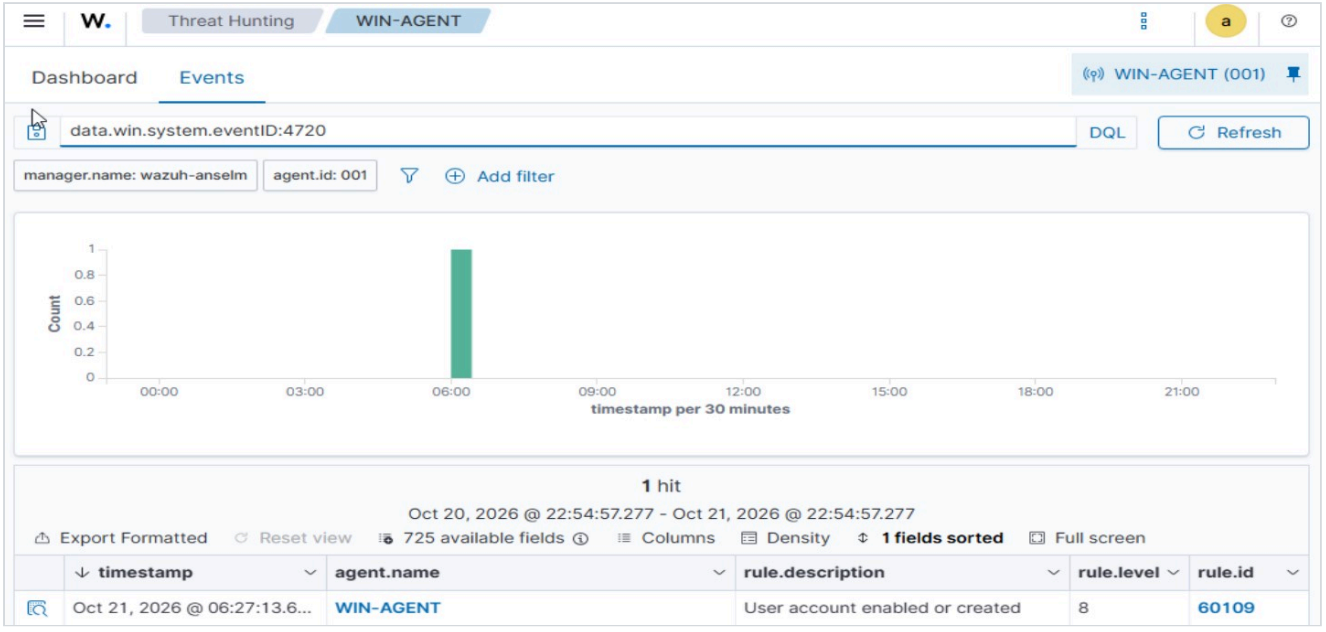


Figure 3: the same event in Wazuh: rule 60109, level 8.

The account was made an administrator

We discovered that `svc_backup` was added to the Administrators group at 14:27:47. Event 4732 returned two alerts: rule 60154, "Administrators Group Changed", at **level 12**, and rule 60170, "Users Group Changed", at level 5. The level 12 alert is the clearest escalation signal in Run 1.

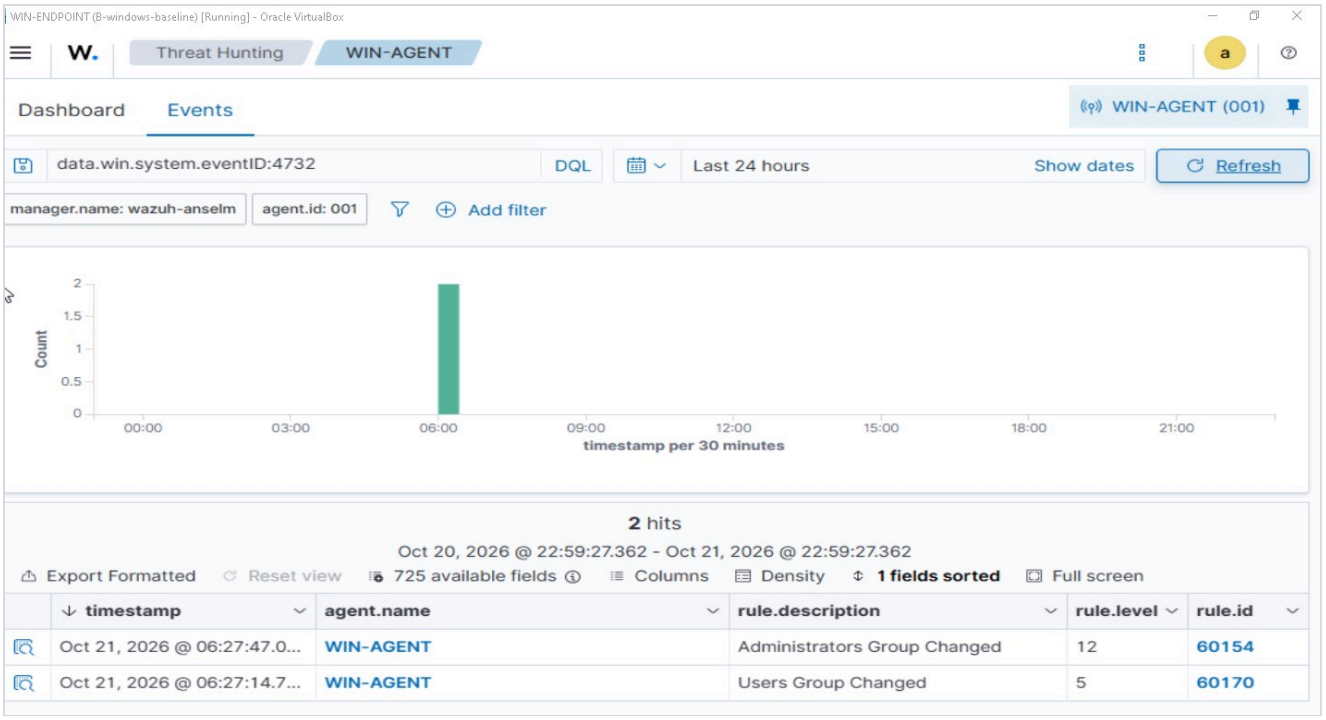


Figure 4: rules 60154 (level 12) and 60170 (level 5).

A new Linux user appeared

We discovered that a new user, `testsvc3`, was created on the Ubuntu server at 14:30:58. Wazuh raised rule 5902, "New user added to the system", and rule 5901, "New group added to the system", both level 8, plus rule 5402 for the sudo command that ran it.

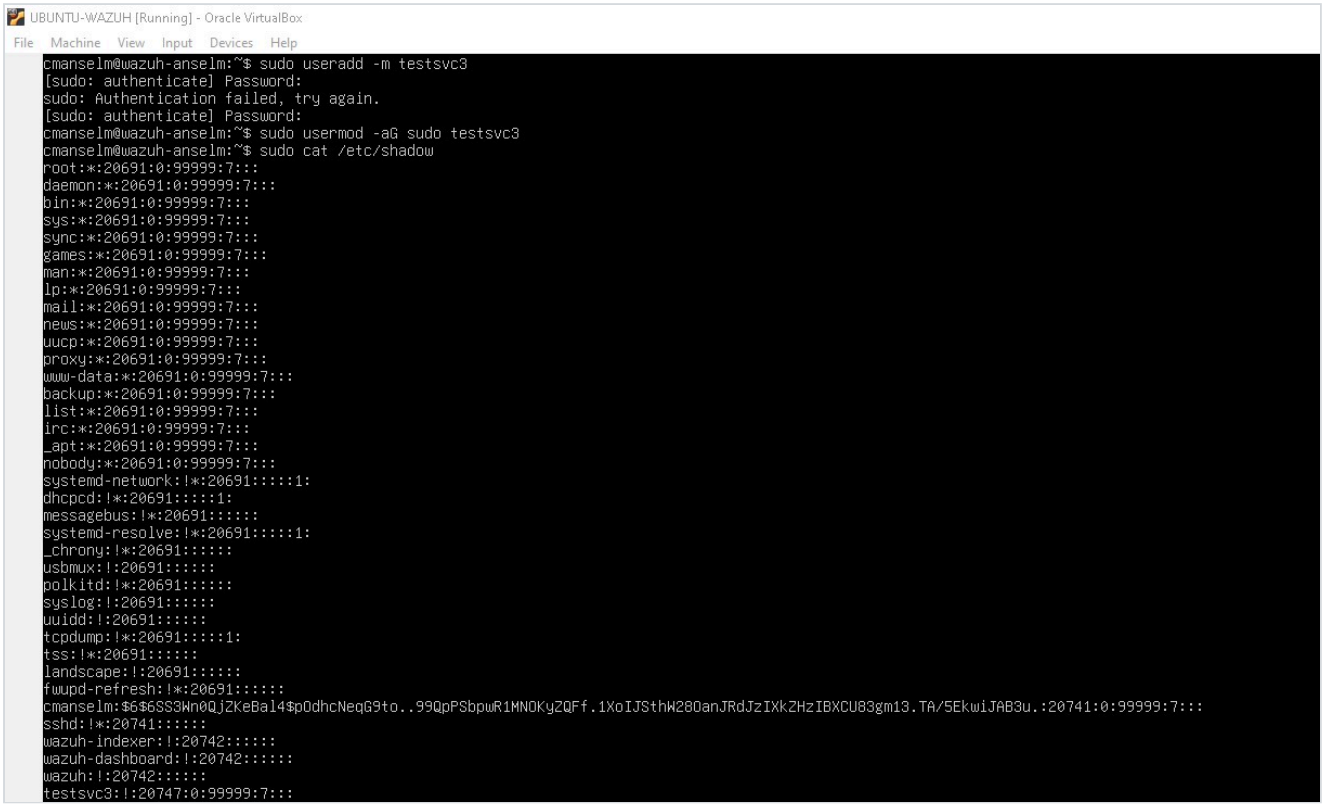


Figure 5: the commands we ran on Ubuntu: useradd, usermod and cat /etc/shadow.

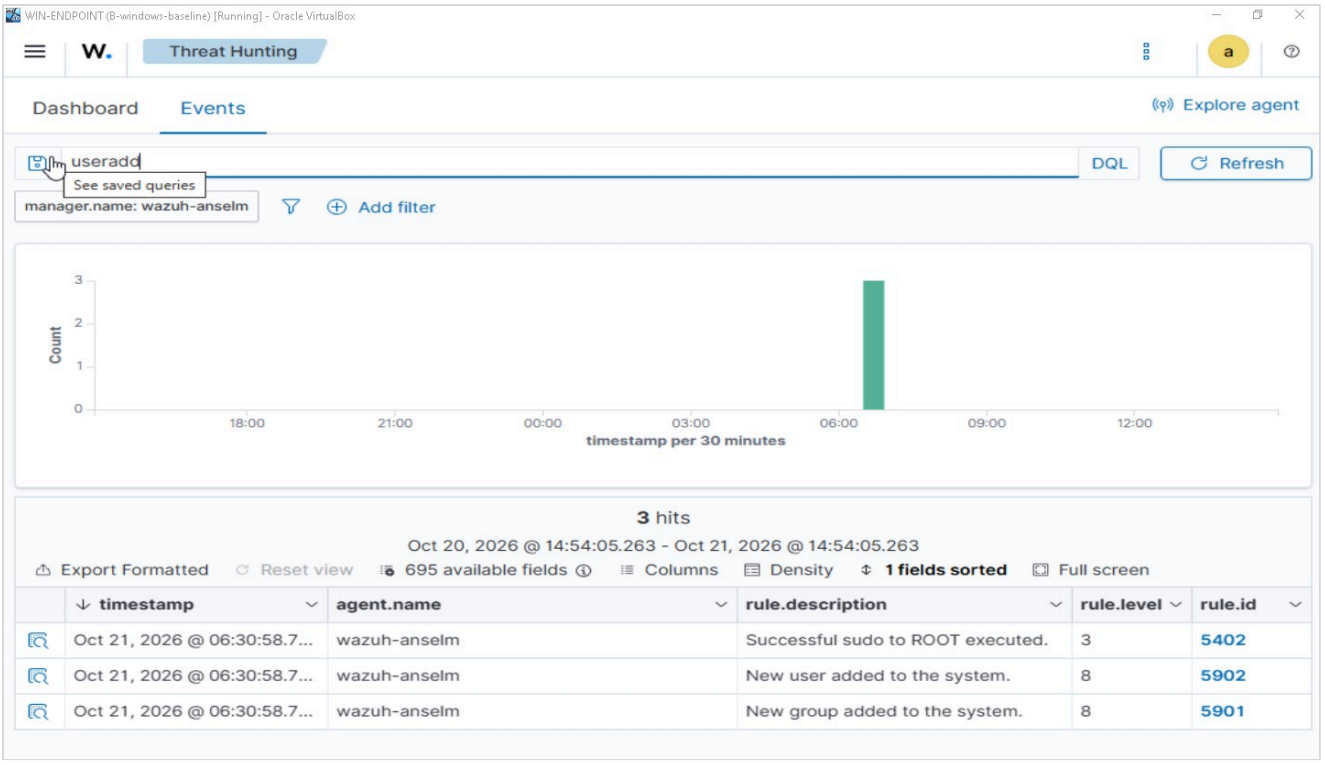


Figure 6: searching `useradd` in Wazuh returns rules 5402, 5902 and 5901.

The later Linux steps left only a generic trace

We discovered that the next two steps, `usermod -aG sudo` (14:31:26) and `sudo cat /etc/shadow` (14:31:40), appear only as rule 5402, "Successful sudo to ROOT executed", level 3. We found them by keyword search, because sudo writes the full command into the log. Nothing in Wazuh said that a user had joined the sudo group or that the password file had been read.

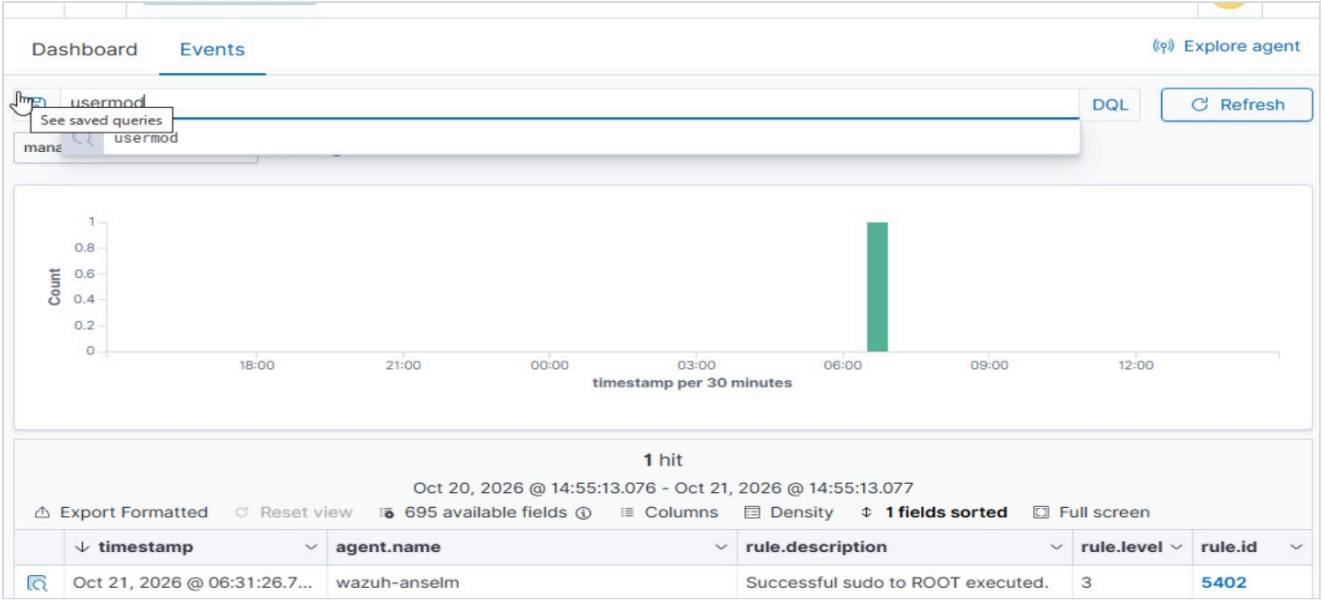


Figure 7: `usermod` returns one hit, the generic sudo alert.

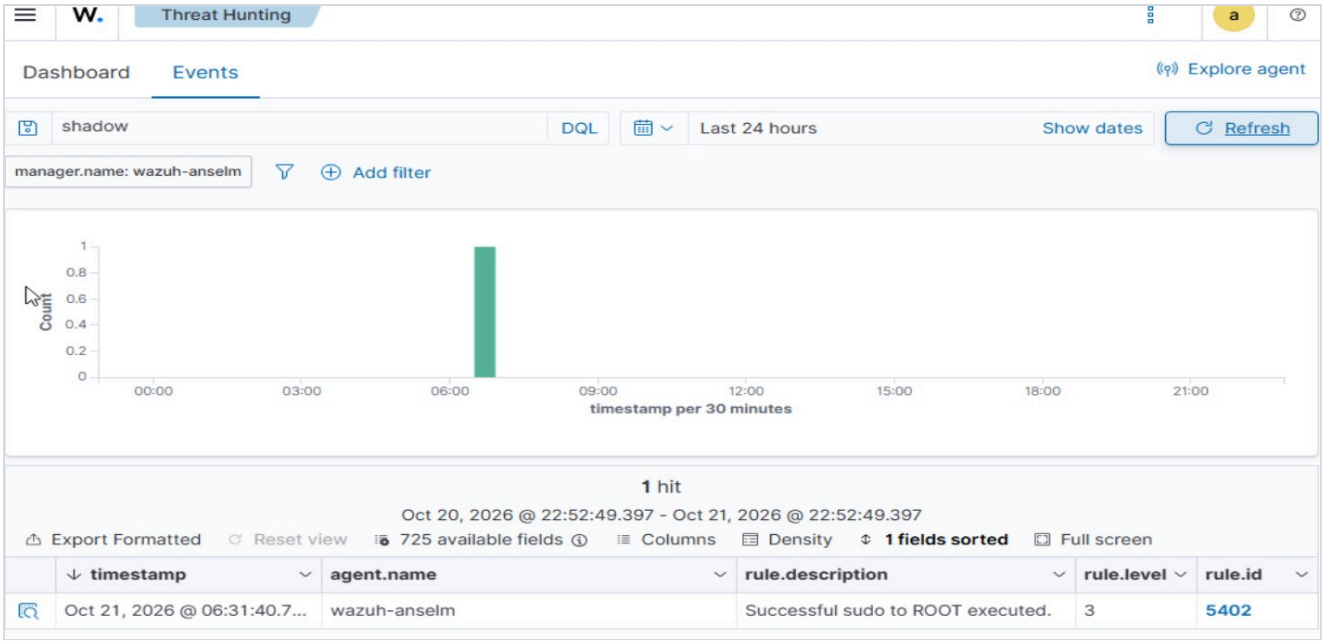


Figure 8: `shadow` returns one hit, the same generic alert.

A wrong sudo password was caught

We discovered that Run 1 also caught the wrong sudo passwords. Wazuh raised rule 5557, "unix_chkpwd: Password check failed" (level 5), at 14:30:48 and again at 14:32:16, with the raw log line "password check failed for user (cmanselm)". Each one came with a second alert at the same second.

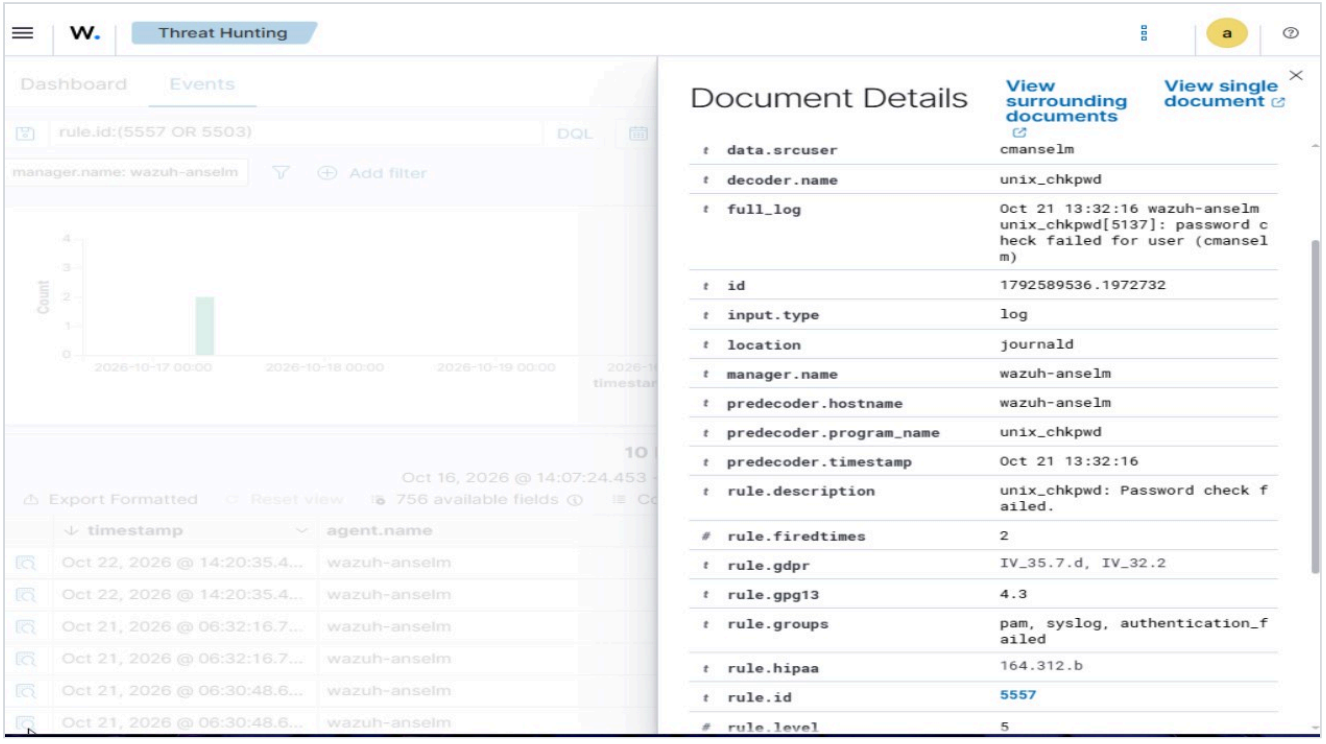


Figure 8a: rule 5557 in Run 1: level 5, with the raw log line.

Default process logs do not show what was typed

We discovered that the default Windows process event (event 4688, rule 67027) records that a process started, who started it and its parent, but the Process Command Line field is empty in our

lab. The example below is a background process from 23 Oct (WmiPrvSE.exe), shown to illustrate what default logging gives. An analyst sees that a program ran, but not the command that was typed.

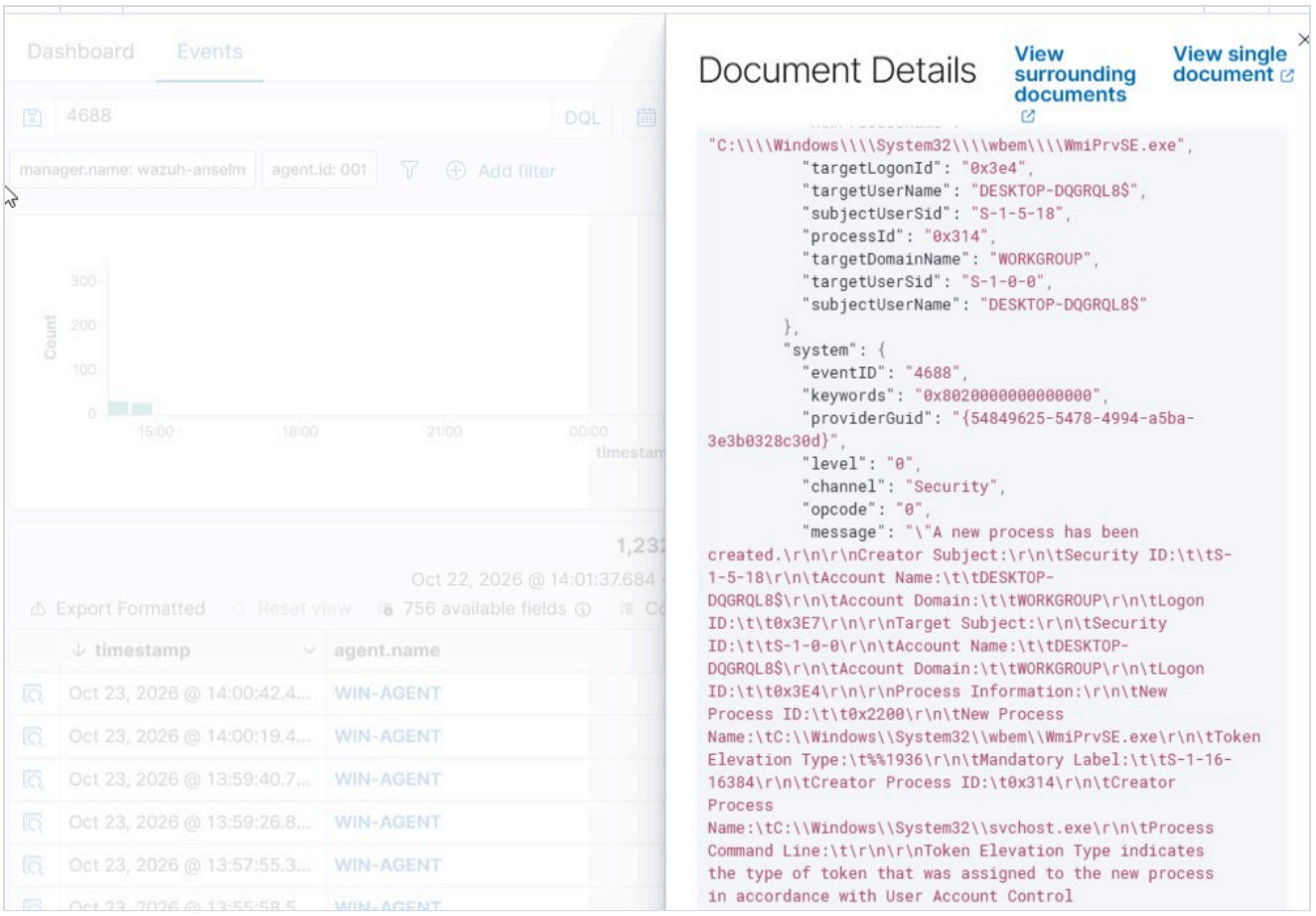


Figure 8b: event 4688: Process Command Line is empty.

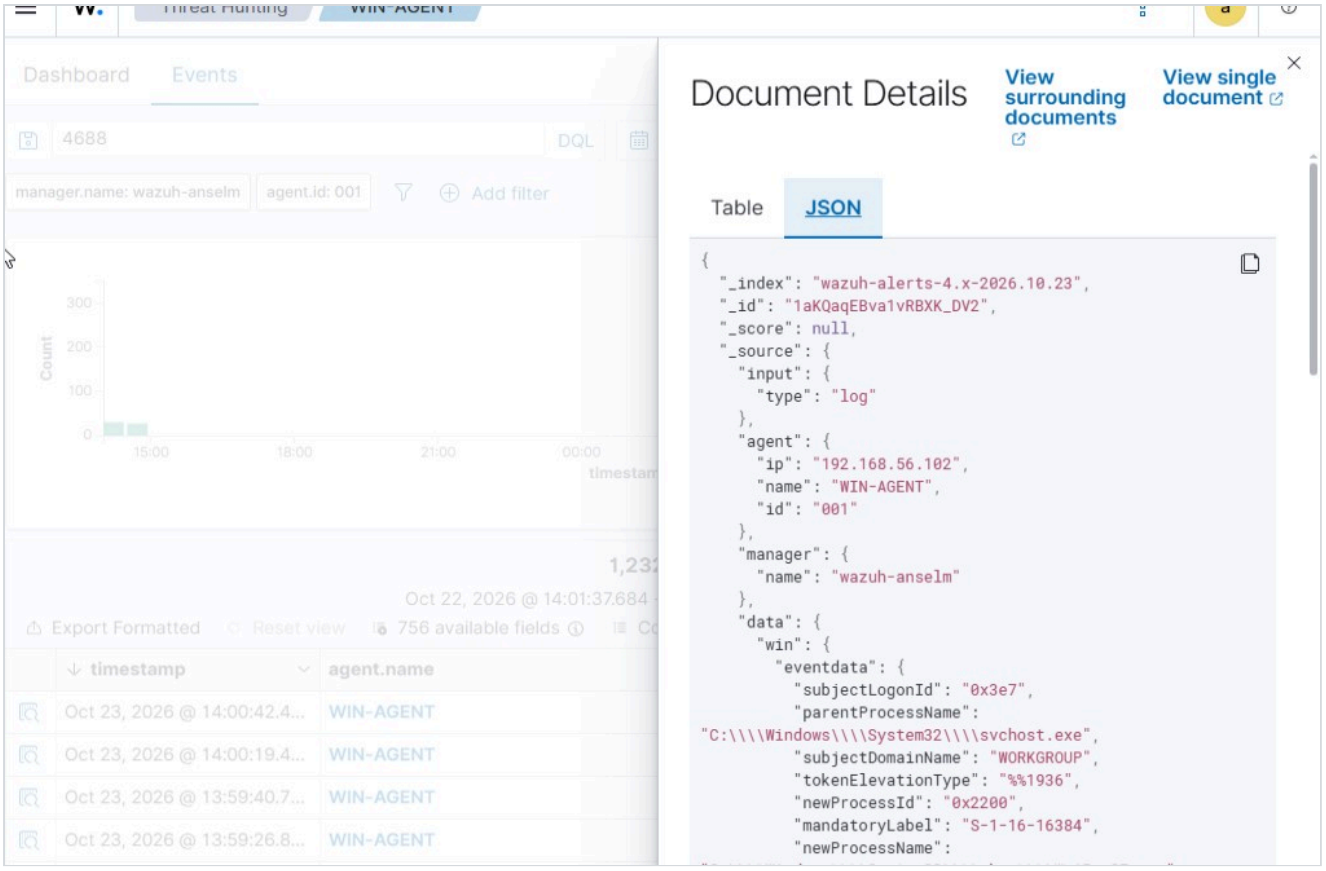


Figure 8c: the same event in JSON, with the parent process and no command line.

Wazuh mapped the activity to MITRE ATT&CK

We discovered that Wazuh grouped the activity under MITRE ATT&CK techniques: Valid Accounts, Sudo and Sudo Cache, Password Guessing, and Disable or Modify Tools.

Run 2: what Wazuh showed with Sysmon and Auditd

We repeated the attack on 22 Oct with new names (`svc_backup2` on Windows, `testsvc4` on Ubuntu), after installing Sysmon on Windows and Auditd on Ubuntu. Auditd watches the identity files (`/etc/passwd`, `/etc/shadow`, `/etc/group`), the sudoers file, and every command that runs.

```
cmanselm@wazuh-anselm:~$ sudo /var/ossec/bin/wazuh-analysisd -t
cmanselm@wazuh-anselm:~$ sudo systemctl restart wazuh-manager
cmanselm@wazuh-anselm:~$ sudo systemctl is-active wazuh-manager
active
cmanselm@wazuh-anselm:~$ sudo tee -a /var/ossec/etc.conf <<'EOF'
> <ossec_config>
>   <localfile>
>     <log_format>audit</log_format>
>      <location>/var/log/audit/audit.log</location>
>   </localfile>
> </ossec_config>
> EOF
<ossec_config>
  <localfile>
    <log_format>audit</log_format>
     <location>/var/log/audit/audit.log</location>
  </localfile>
</ossec_config>
cmanselm@wazuh-anselm:~$ sudo tail -9 /var/ossec/etc/ossec.conf
    <location>/var/ossec/logs/active-responses.log</location>
  </localfile>

  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/dpkg.log</location>
  </localfile>

</ossec_config>
cmanselm@wazuh-anselm:~$ sudo /var/ossec/bin/wazuh-analysisd -t
cmanselm@wazuh-anselm:~$ sudo systemctl restart wazuh-manager
cmanselm@wazuh-anselm:~$ sudo systemctl is-active wazuh-manager
active
cmanselm@wazuh-anselm:~$ sudo auditctl -l
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p rwa -k identity
-w /etc/group -p wa -k identity
-w /etc/sudoers -p wa -k sudo_changes
-a always,exit -F arch=b64 -S execve -F key=exec_cmds
cmanselm@wazuh-anselm:~$
```

Figure 10: the five Auditd rules loaded on the Ubuntu server.

Sysmon showed what the attacker typed

We discovered that Wazuh now shows the command line. For the `net user svc_backup2` command, the Sysmon event gives the command line `C:\Windows\system32\net1 user svc_backup2`, the parent process `net.exe` with the same arguments, the SHA256 hash of the program, and integrity level High. In Run 1 the process alerts (rule 67027) did not give us a command line we could confirm.

```
PS C:\Windows\system32> net user svc_backup2
User name                    svc_backup2
Full Name
Comment
User's comment
Country/region code          000 (System Default)
Account active               Yes
Account expires              Never

Password last set            10/22/2026 10:16:00 PM
Password expires             12/3/2026 10:16:00 PM
Password changeable          10/22/2026 10:16:00 PM
Password required            Yes
User may change password     Yes

Workstations allowed         All
Logon script
User profile
Home directory
Last logon                   Never

Logon hours allowed          All

Local Group Memberships      *Administrators       *Users

Global Group memberships     *None
The command completed successfully.

PS C:\Windows\system32>
```

Figure 11: `svc_backup2`, created at 22:16:00, a member of Administrators and Users.

Table JSON

t	_index	wazuh-alerts-4.x-2026.10.22
t	agent.id	001
t	agent.ip	192.168.56.102
t	agent.name	WIN-AGENT
t	data.win.eventdata.commandLine	C:\\Windows\\system32\\net1 user svc_backup2
t	data.win.eventdata.company	Microsoft Corporation
t	data.win.eventdata.currentDirectory	C:\\Windows\\system32\\
t	data.win.eventdata.description	Net Command
t	data.win.eventdata.fileVersion	10.0.19041.3636 (WinBuild.160101.0800)
t	data.win.eventdata.hashes	SHA256=E62071AA18768DD88ACAF97FA7B1F2FEC9FCCE89736C1EE9A800699328D196EA
t	data.win.eventdata.image	C:\\Windows\\System32\\net1.exe
t	data.win.eventdata.integrityLevel	High
t	data.win.eventdata.logonGuid	{e4848489-cf4a-6ada-00000000}

Figure 12: the Sysmon event: command line, company, hash and integrity level.



Figure 13: the same event: parent command line, parent image and parent process ID.

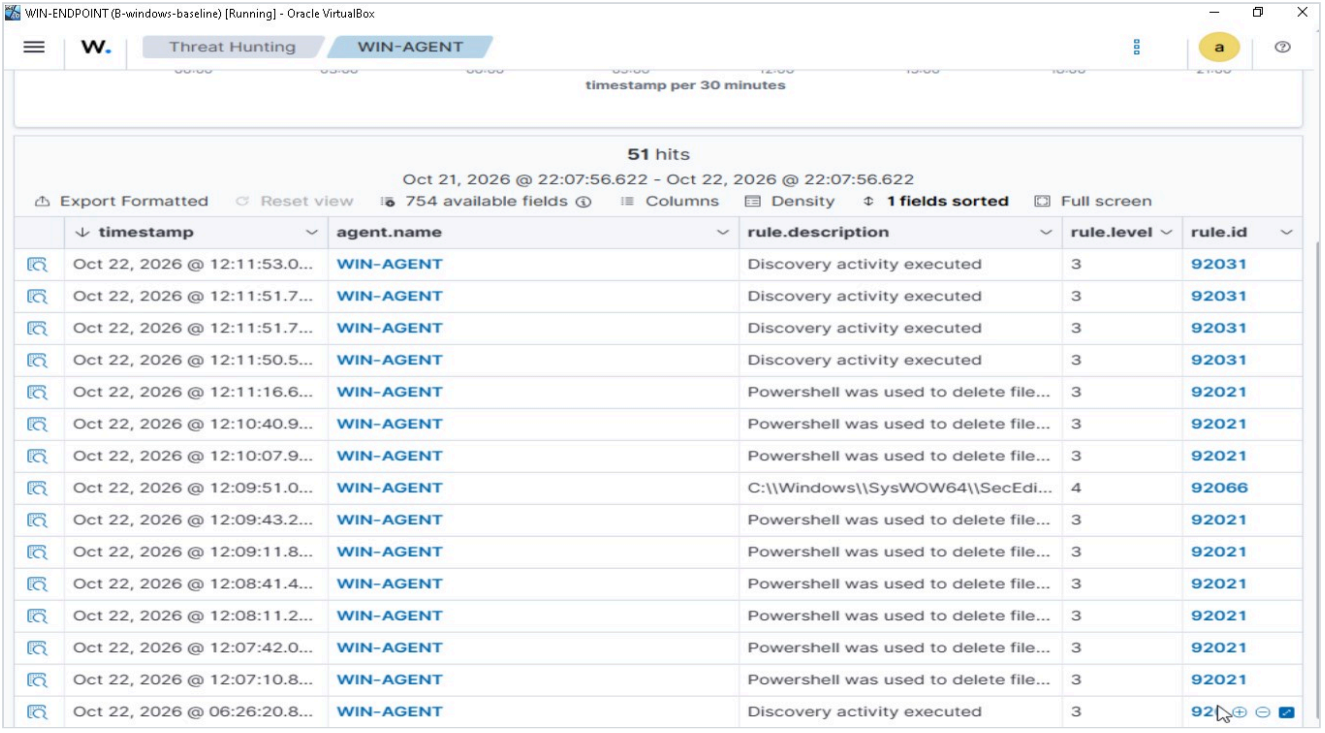


Figure 14: Sysmon-based alerts on WIN-AGENT (rules 92031, 92021 and 92066, levels 3 to 4).

The Windows account and group change were logged again

We discovered that the Run 2 account was logged too. The search `rule.id:(60109 OR 60154)` returned three alerts for WIN-AGENT at 22:16 (machine clock). Event 4722 shows `svc_backup2`

enabled by the user `anselm`, and event 4732 shows a member added to the Administrators group. So Windows gave us the same account and group-change alerts as in Run 1.

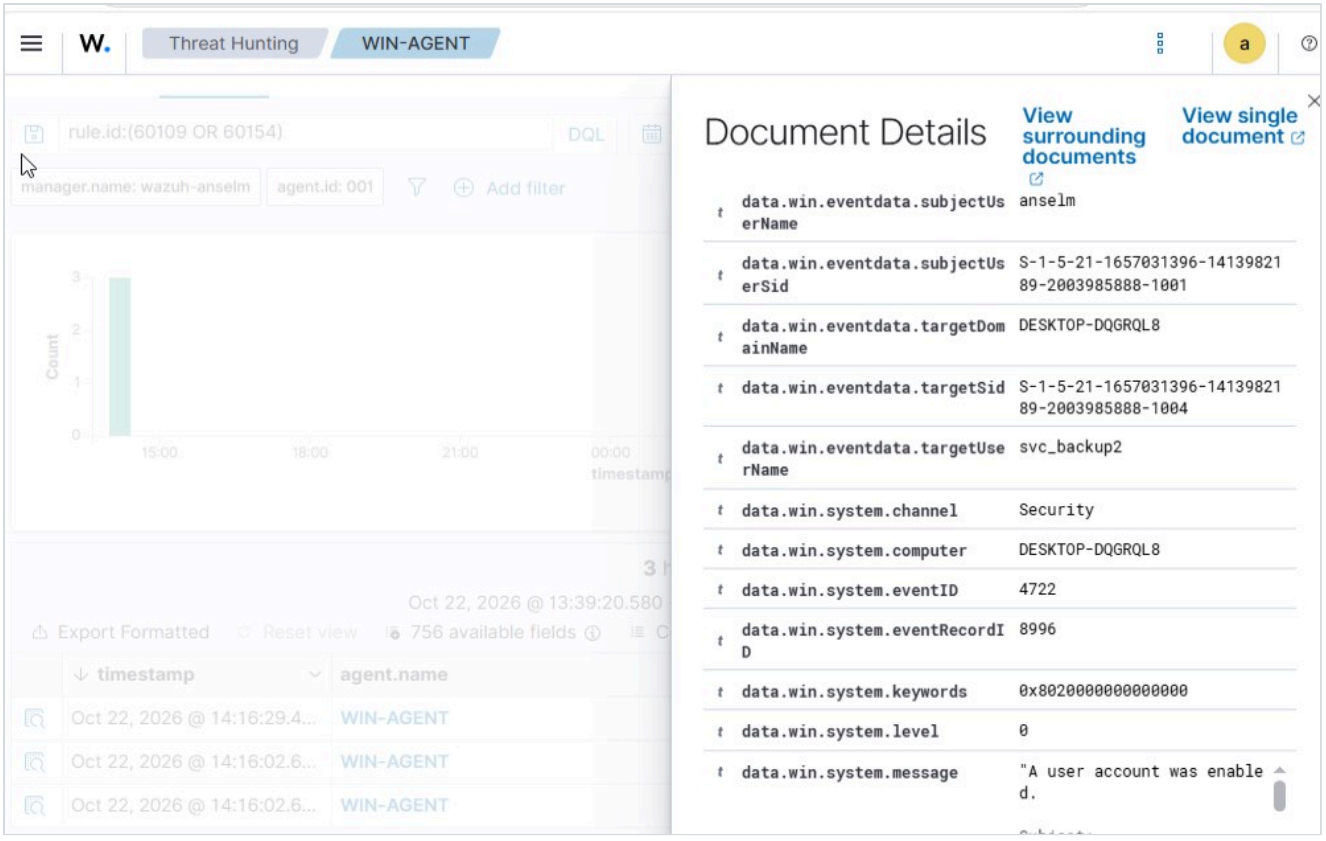


Figure 14a: event 4722 for `svc_backup2`, performed by `anselm`.

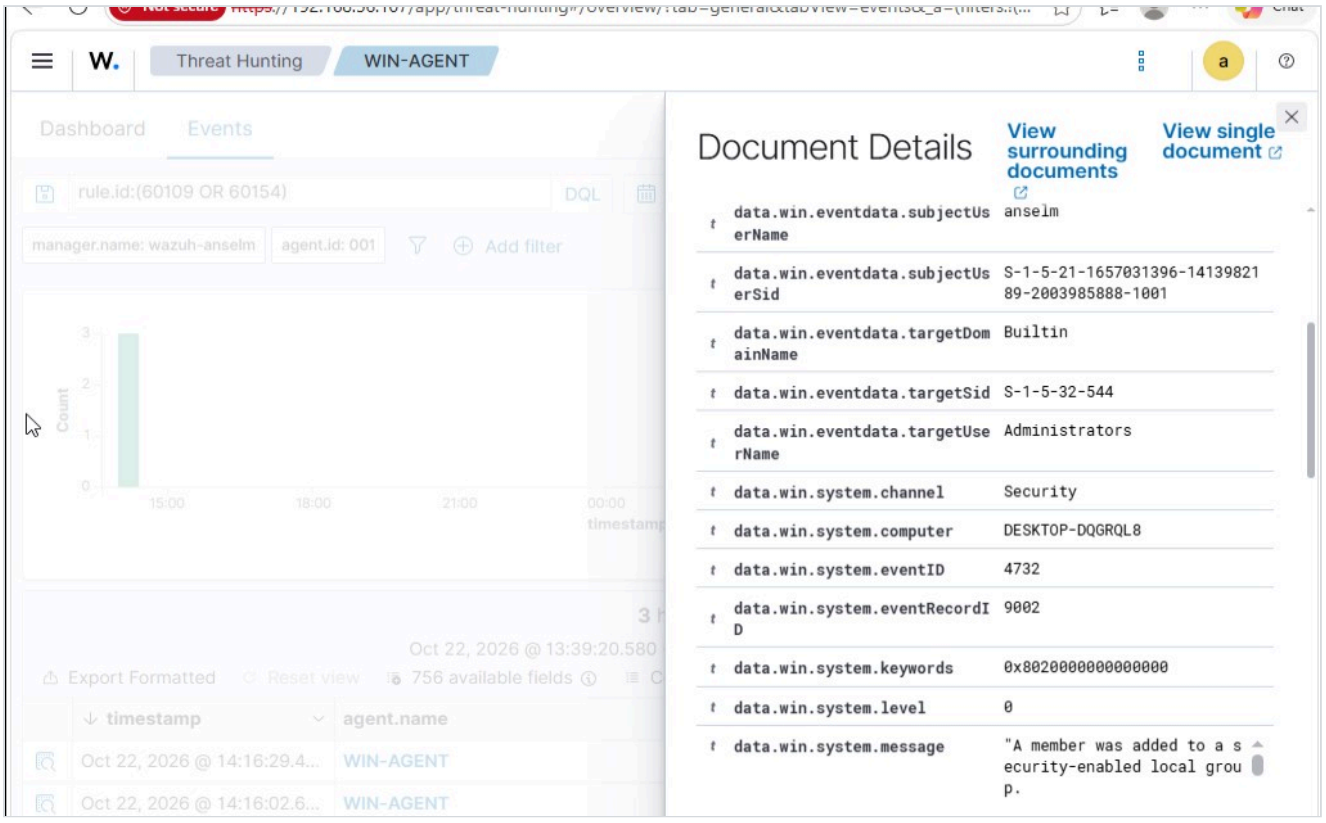


Figure 14b: event 4732, a member added to the Administrators group.

Auditd raised an alert on the password file

We discovered that access to `/etc/shadow` now raises our custom rule 100100, "Auditd: identity file touched", at level 8. The alert shows the file name, the audit key `identity`, the program (`unix_chkpwd`, which sudo uses to check the password), the user ID (auid 1000) and the terminal (tty1).



Figure 15: rule 100100 on `/etc/shadow`, with the program, user ID and key.

Wazuh's own processes trigger the same rule

We discovered that rule 100100 also fires when Wazuh's own `wazuh-syscheckd` and `wazuh-modulesd` read `/etc/shadow`. Six alerts at 21:34 to 21:35 came from this. In a real SOC that noise has to be filtered out, or it hides the alerts that matter.

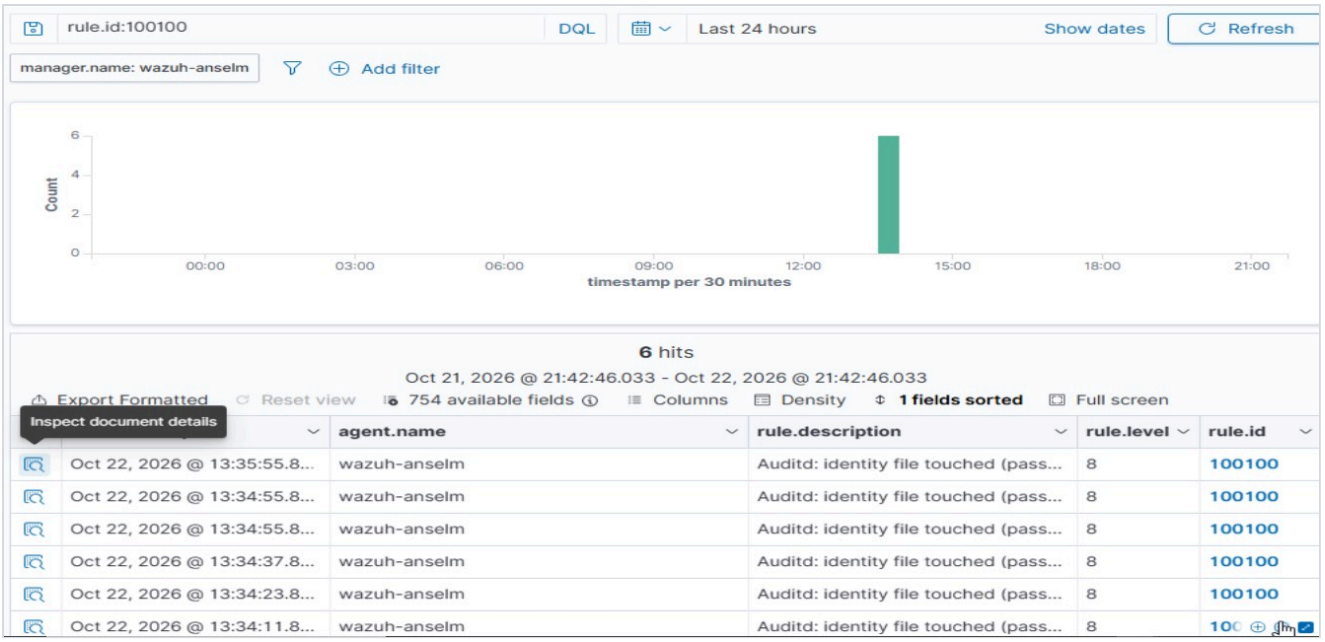


Figure 16: six alerts of rule 100100 in two minutes.

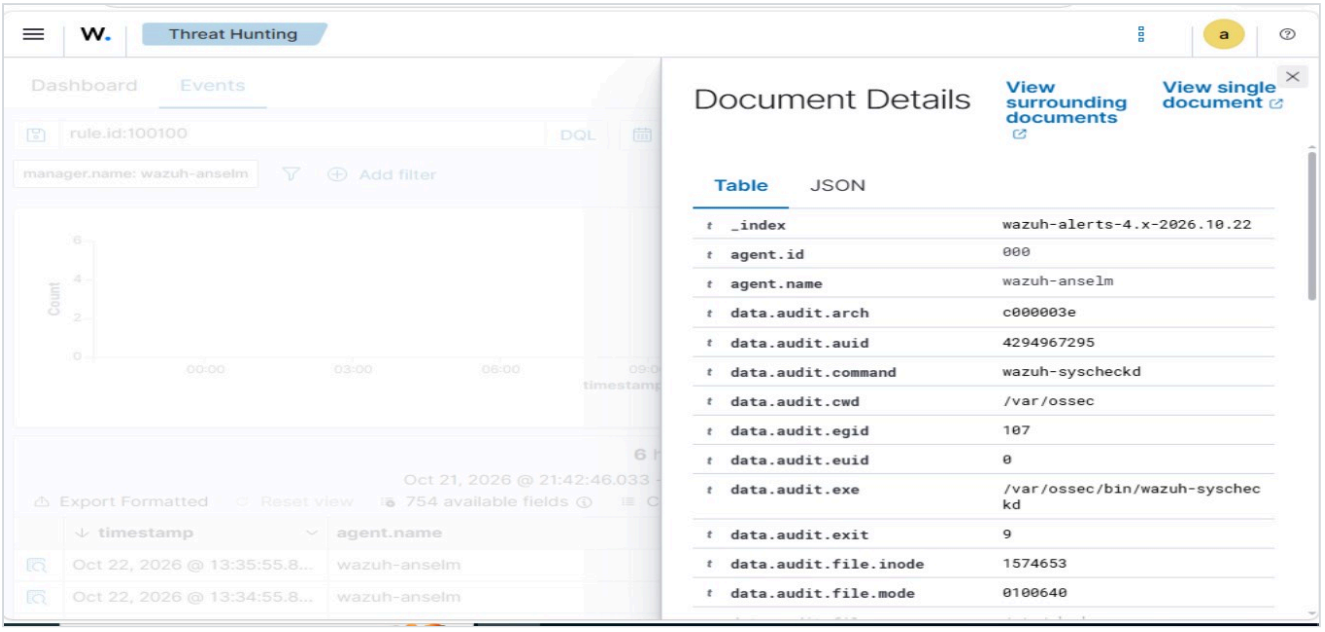


Figure 17: one of them was caused by `wazuh-syscheckd`.

Auditd also saw the sudo group change

We discovered that Auditd recorded the `usermod` command. Searching `rule.id:100100 AND data.audit.exe:"/usr/sbin/usermod"` returned five alerts at 22:19:10 (machine clock). The detail shows the program `/usr/sbin/usermod`, the key `identity`, a successful call, and the terminal `pts0`. We did not capture an equivalent hit for `useradd`.

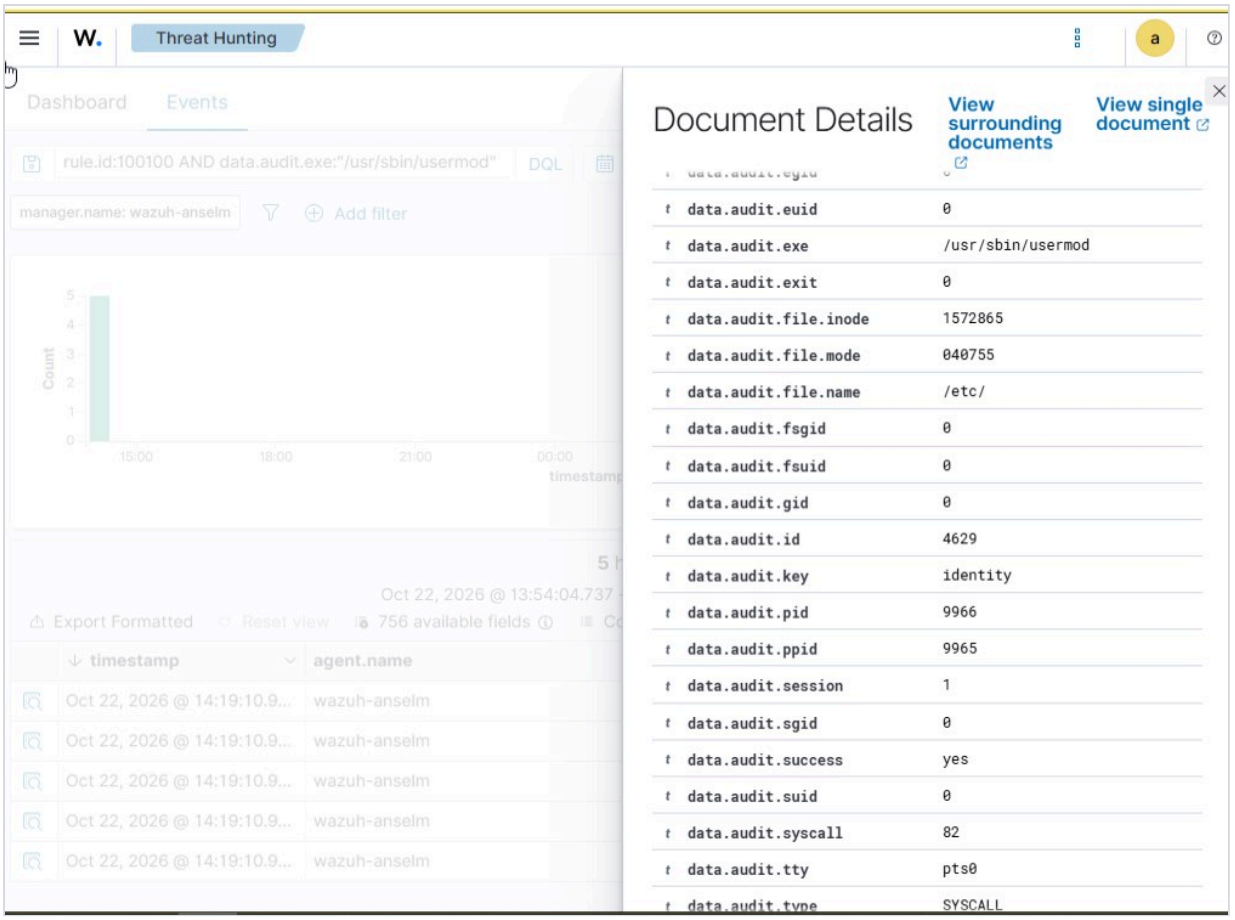


Figure 17a: rule 100100 for `/usr/sbin/usermod`.

A wrong password was caught by default rules

We discovered that the wrong sudo password at 22:20:35 raised rule 5557, "unix_chkpwd: Password check failed", and rule 5503, "PAM: User login failed", both level 5. These are default Wazuh rules, so this detection does not depend on Auditd.

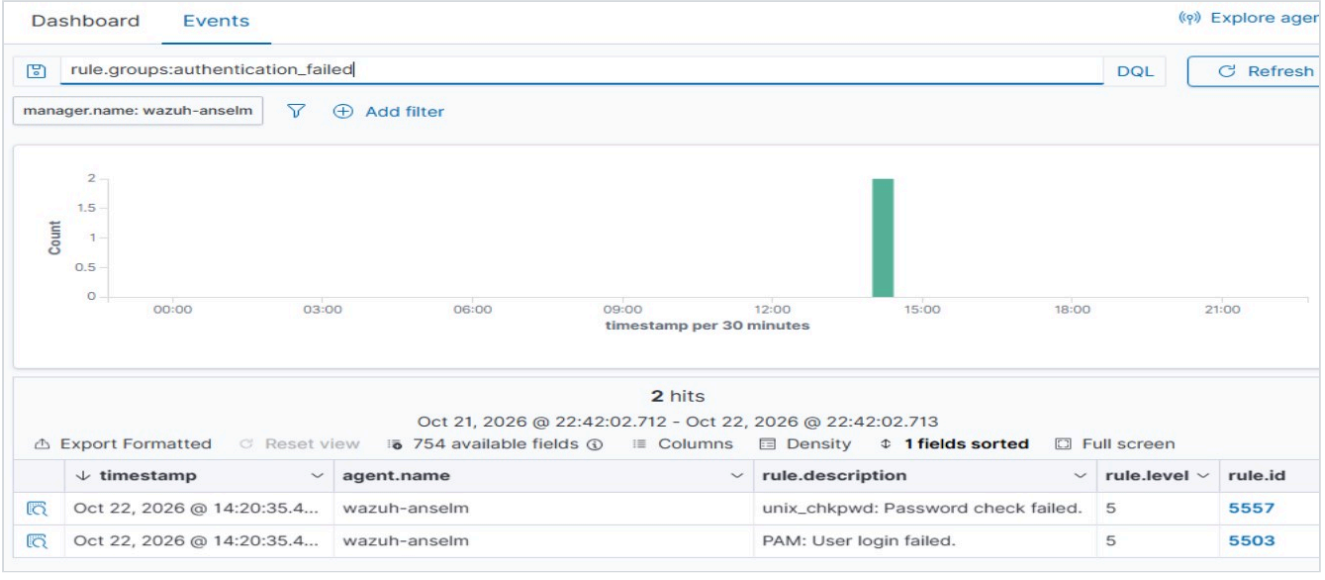


Figure 18: the two failed-authentication alerts.

The overview after Run 2

We discovered that the Threat Hunting overview for the 24 hours after Run 2 shows 1,994 alerts, 5 of them at level 12 or above, 3 authentication failures and 32 successes. Its MITRE ATT&CK panel lists techniques such as File Deletion, Valid Accounts, Account Discovery, PowerShell and Account Manipulation.

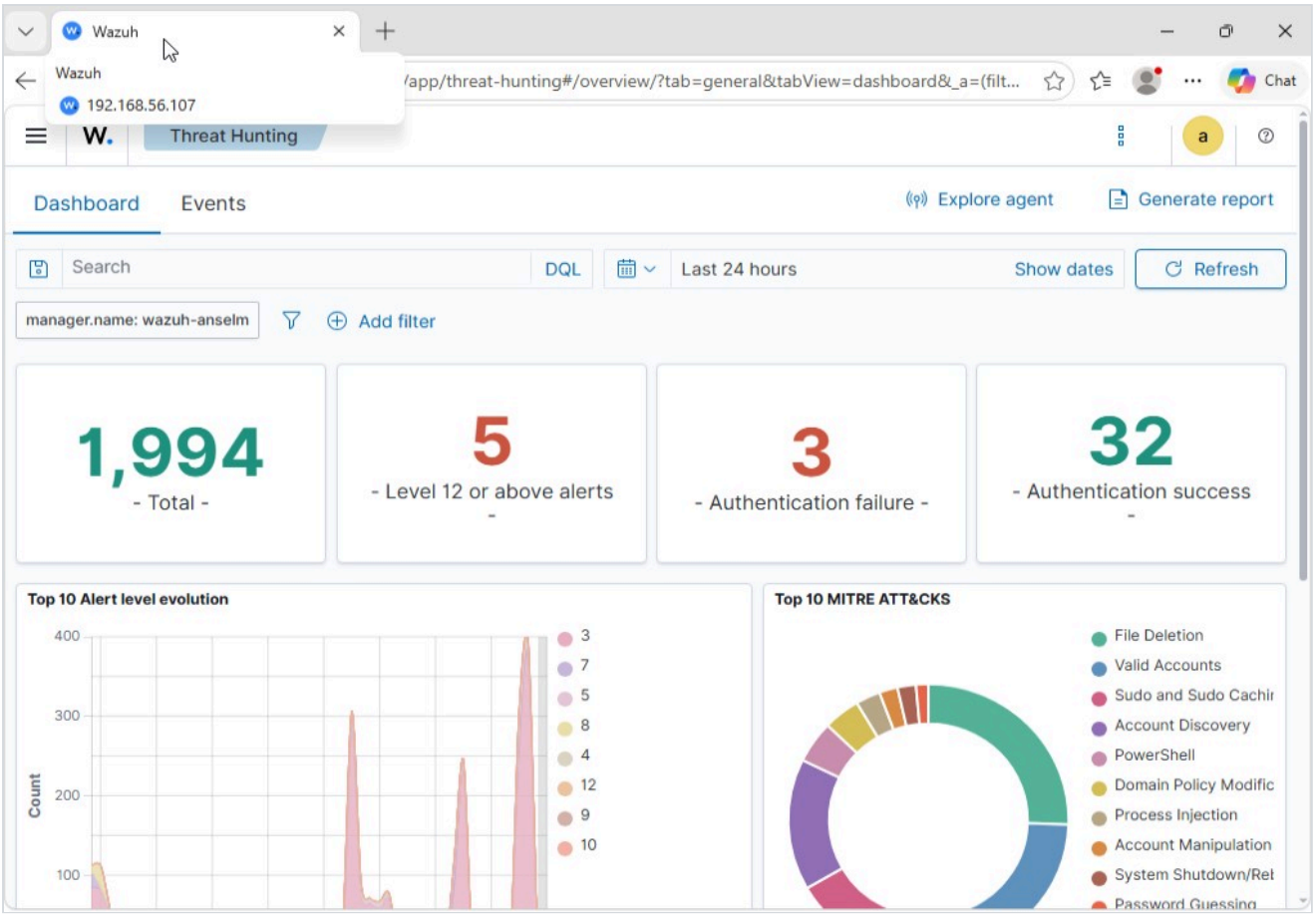


Figure 18a: the overview, taken on 23 Oct.

Visibility gaps and comparison

We discovered that Sysmon and Auditd each close one real gap, but neither closes everything. Default logging already caught the account creation and the admin change on both machines.

Attacker step	Run 1 (default logging)	Run 2 (Sysmon and Auditd)	Gap closed?
Windows account created	Alerted, rule 60109 (level 8)	Alerted again (event 4722), plus the Sysmon command line	Already visible
Added to Administrators	Alerted, rule 60154 (level 12)	Alerted again (event 4732)	Already visible
What was typed: command line, parent, hash	Not in the 4688 process event	Shown by Sysmon	**Yes**

Linux user created	Alerted, rules 5902 and 5901 (level 8)	No Auditd hit captured for `useradd`	Not shown
Added to the sudo group	Only the generic rule 5402 (level 3)	Rule 100100 (level 8) for `usermod`	**Yes**
`/etc/shadow` read	Only the generic rule 5402 (level 3)	Rule 100100 (level 8), with program, user ID and terminal	**Yes**, with noise
Wrong sudo password	Default rules 5557 and 5503 (level 5)	Same default rules	Not needed

Recommendations

1. **Run Sysmon on Windows endpoints.** It adds the command line, the parent process and the file hash that default logging lacks.
2. **Run Auditd on Linux servers** and watch the identity files and the sudoers file, using clear key names such as `identity` and `sudo_changes`.
3. **Filter Wazuh's own processes out of rule 100100** (`wazuh-syscheckd` and `wazuh-modulesd`), so real activity is not buried.
4. **Keep the level 12 alert for Administrators group changes** (rule 60154) as a priority alert, and add one for sudo group changes on Linux.
5. **Test every configuration change** with `wazuh-analysisd -t` before restarting the manager, and give the indexer enough memory.
6. **Fix the dashboard time zone** so alert times match the machine clocks.

Conclusion

We discovered that the answer is **PARTIALLY**: default logging already shows that accounts were created and escalated, and Sysmon and Auditd add the detail an analyst needs to understand what happened, but the picture is not complete.

- **What default logging gives:** alerts for the new accounts and the Administrators change (level 12), and a generic sudo alert. It gives no command line and no file name.
- **What Sysmon adds:** the command line, the parent process, the hash and the integrity level on Windows.
- **What Auditd adds:** alerts on the identity files and on `usermod`, with the program, user ID and terminal, once a custom rule exists. The rule also fires on Wazuh's own processes and needs filtering.
- **What we did not show:** an Auditd hit for `useradd`, and the filtered version of rule 100100.

Filtering the noise and showing `useradd` in Auditd are the next steps towards a YES.